

Security Vulnerability Report

Executive Summary

- Scan Overview:**
- Scan ID: 803a733f-2f8a-434a-b2d4-646d90dfb5eb
 - Date: 2025-11-10T06:07:16.657Z
 - Files Scanned: 1
 - Total Vulnerabilities: 14

- Severity Breakdown:**
- Critical: 0
 - High: 4
 - Medium: 8
 - Low: 2

Vulnerability Summary

File	Line	Type	Severity	CWE
files-1762754836685-166069909.py	38	subprocess_popen_with_shell_equals_true	High	CWE-78
files-1762754836685-166069909.py	81	hashlib	High	CWE-327
files-1762754836685-166069909.py	25	sqlalchemy:raw_compiler	High	CWE-89
files-1762754836685-166069909.py	38	subprocess_popen_with_shell_equals_true	High	CWE-78
files-1762754836685-166069909.py	24	hardcoded_sql_expressions	Medium	CWE-89
files-1762754836685-166069909.py	46	blacklist	Medium	CWE-78
files-1762754836685-166069909.py	55	blacklist	Medium	CWE-502
files-1762754836685-166069909.py	63	hardcoded_tmp_directory	Medium	CWE-377
files-1762754836685-166069909.py	25	format_string_with_untrusted_data	High	CWE-89
files-1762754836685-166069909.py	46	eval	High	CWE-95
files-1762754836685-166069909.py	55	avoid-pickle	Medium	CWE-502
files-1762754836685-166069909.py	81	md5-used-as-password	Medium	CWE-327
files-1762754836685-166069909.py	31	blacklist	Low	CWE-78
files-1762754836685-166069909.py	49	blacklist	Low	CWE-502

Detailed Vulnerability Findings

Finding 1: subprocess_popen_with_shell_equals_true

File: files-1762754836685-166069909.py

Line: 38

Severity: High

CWE ID: CWE-78

Detection Tool: Bandit

Confidence: 90%

Description:

subprocess call with shell=True identified, security issue. Recommendation: Review the code for security implications and apply secure coding practices.

Code Snippet:

```
37     # UNSAFE: shell=True with concatenated user input
38     subprocess.call(cmd, shell=True)
39
```

Finding 2: hashlib

File: files-1762754836685-166069909.py

Line: 81

Severity: High

CWE ID: CWE-327

Detection Tool: Bandit

Confidence: 90%

Description:

Use of weak MD5 hash for security. Consider usedforsecurity=False Recommendation: Use secure hash functions. Prefer SHA-256 or better.

Code Snippet:

```
80     """
81     return hashlib.md5(password.encode()).hexdigest()
82
```

Finding 3: sqlalchemy-execute-raw-query

File: files-1762754836685-166069909.py

Line: 25

Severity: High

CWE ID: CWE-CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

Detection Tool: Semgrep

Confidence: 85%

Description:

Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL Injection. In order to execute raw query safely, prepared statement should be used.

SQLAlchemy provides TextualSQL to easily used prepared statement with named parameters. For

complex SQL composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy ORM will be a better option. Recommendation: Use parameterized queries (prepared statements) instead of string concatenation. Example: ```python
cursor.execute("SELECT * FROM users WHERE id = %s", [user_id]) ```

Code Snippet:

```
23:      # UNSAFE: string formatting of user input
24:      query = "SELECT id, username FROM users WHERE username LIKE '%{}%';".format(q)
25:      cur = conn.execute(query)
26:      rows = cur.fetchall()
27:      conn.close()
```

Finding 4: subprocess-shell-true

File: files-1762754836685-166069909.py

Line: 38

Severity: High

CWE ID: CWE-CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

Detection Tool: Semgrep

Confidence: 85%

Description:

Found 'subprocess' function 'call' with 'shell=True'. This is dangerous because this call will spawn the command using a shell process. Doing so propagates current shell settings and variables, which makes it much easier for a malicious actor to execute commands. Use 'shell=False' instead. Recommendation: Review the code for security implications and consider using secure coding practices appropriate for your use case.

Code Snippet:

```
36:      cmd = "ping -c 1 " + target
37:      # UNSAFE: shell=True with concatenated user input
38:      subprocess.call(cmd, shell=True)
39:
40: # ===== Eval (RCE) =====
```

Finding 5: hardcoded_sql_expressions

File: files-1762754836685-166069909.py

Line: 24

Severity: Medium

CWE ID: CWE-89

Detection Tool: Bandit

Confidence: 50%

Description:

Possible SQL injection vector through string-based query construction. Recommendation: Use parameterized queries or ORM methods instead of string formatting for SQL queries.

Code Snippet:

```
23      # UNSAFE: string formatting of user input
24      query = "SELECT id, username FROM users WHERE username LIKE '%{}%';".format(q)
25      cur = conn.execute(query)
```

Finding 6: blacklist

File: files-1762754836685-166069909.py

Line: 46

Severity: Medium

CWE ID: CWE-78

Detection Tool: Bandit

Confidence: 90%

Description:

Use of possibly insecure function - consider using safer `ast.literal_eval`. Recommendation: Avoid using `eval()` as it can execute arbitrary code. Use `ast.literal_eval()` for safe evaluation.

Code Snippet:

```
45     # UNSAFE: eval on raw user input
46     return eval(user_expr)
47
```

Finding 7: blacklist

File: files-1762754836685-166069909.py

Line: 55

Severity: Medium

CWE ID: CWE-502

Detection Tool: Bandit

Confidence: 90%

Description:

Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible security issue. Recommendation: Avoid using pickle for deserialization. Use safer alternatives like JSON.

Code Snippet:

```
54     # UNSAFE: pickle can execute arbitrary code during unpickling
55     obj = pickle.loads(data_bytes)
56     return obj
```

Finding 8: hardcoded_tmp_directory

File: files-1762754836685-166069909.py

Line: 63

Severity: Medium

CWE ID: CWE-377

Detection Tool: Bandit

Confidence: 70%

Description:

Probable insecure usage of temp file/directory. Recommendation: Use secure temporary file creation methods like `tempfile.mkstemp()` instead of hardcoding paths.

Code Snippet:

```
62     """
63     path = Path("/tmp") / filename # predictable path + no sanitization
64     with open(path, "w") as f:
```

Finding 9: formatted-sql-query

File: files-1762754836685-166069909.py

Line: 25

Severity: Medium

CWE ID: CWE-CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

Detection Tool: Semgrep

Confidence: 85%

Description:

Detected possible formatted SQL query. Use parameterized queries instead. Recommendation:

Use parameterized queries (prepared statements) instead of string concatenation. Example:

```
```python cursor.execute("SELECT * FROM users WHERE id = %s", [user_id]) ```
```

### Code Snippet:

```
23: # UNSAFE: string formatting of user input
24: query = "SELECT id, username FROM users WHERE username LIKE '{}{}';".format(q)
25: cur = conn.execute(query)
26: rows = cur.fetchall()
27: conn.close()
```

## Finding 10: eval-detected

**File:** files-1762754836685-166069909.py

**Line:** 46

**Severity:** Medium

**CWE ID:** CWE-CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')

**Detection Tool:** Semgrep

**Confidence:** 85%

### Description:

Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this content can be input from outside the program, this may be a code injection vulnerability. Ensure evaluated content is not definable by external sources. Recommendation: Review the code for security implications and consider using secure coding practices appropriate for your use case.

### Code Snippet:

```
44: """
45: # UNSAFE: eval on raw user input
46: return eval(user_expr)
47:
48: # ===== Insecure Deserialization =====
```

## AI-Generated Recommendations

- Implement parameterized queries for all database operations to prevent SQL injection
- Use environment variables for storing sensitive configuration data
- Implement proper input validation and output encoding
- Use strong cryptographic functions (bcrypt, scrypt, or Argon2) for password hashing
- Regular security code reviews and automated scanning
- Keep dependencies updated and monitor for known vulnerabilities